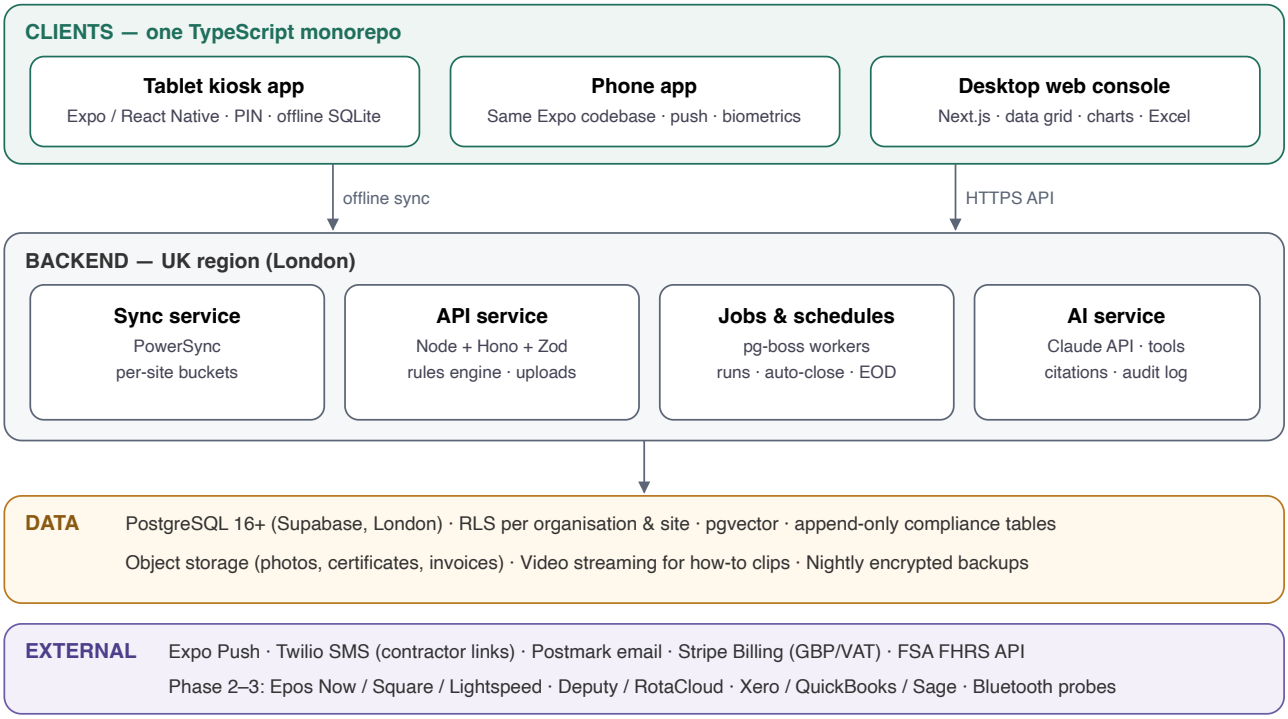
ENGINEERING PLAN · OCTOBER 2026

Tech stack and database plan

Architecture for the UK restaurant operations app: tablet kiosk, phone and desktop console from one TypeScript codebase; offline-first sync; PostgreSQL schema (67 tables, incl. trading / EOD module) with row-level security and append-only compliance records; and the Claude-powered AI layer including Ask your logs.

> This plan turns the UK product plan into an architecture you can build: **one TypeScript codebase** shipping a **kitchen tablet app, a phone app and a desktop web console**, on **PostgreSQL in a London region**, with **offline-first sync** and a traceable **AI layer** (Ask your logs, Ask the handbook, read-the-thermometer, snap-to-setup). Every choice below is driven by four product requirements from the research. Kitchens have bad Wi-Fi, so the tablet must work fully offline. Compliance records must be **tamper-evident** for EHOs. Staff turn over 38–47% a year, so **shared devices with PINs** must be fast. The **£19 entry tier** forces low running costs. The companion file `schema.sql` (v0.2) contains the full PostgreSQL schema: 67 tables, including the **trading / EOD finance module** that replaces Maginhawa's current Netlify form. It has been loaded into a real PostgreSQL 16 server and smoke-tested. The trading calculations, the cash-variance rule and the one-report-per-site-per-day rule all behave as designed. A companion PDF, **System design diagrams**, shows the architecture, flows, lifecycles, sequences and data model. Product-level details such as legal thresholds stay data in the database, never code, because many are still flagged **[unverified]** in the product plan.

The stack at a glance



Layer	Recommended choice	Why this one	Credible alternative
Language & repo	TypeScript everywhere; pnpm workspaces + Turborepo monorepo	One language for tablet, phone, web and API. Shared Zod schemas mean the rules engine and validation are written once	Separate repos (slower for a small team)
Tablet + phone app	**Expo (React Native)** with EAS Build and over-the-air updates	One codebase for iPad, Android tablets, iPhone and Android phones. Tablet and phone get different layouts and navigation from the same code. OTA updates fix kitchens without app-store waits	Flutter (also good, but it splits the language from the web console)

Layer	Recommended choice	Why this one	Credible alternative
Desktop console	**Next.js** (React) + Tailwind + shadcn/ui; AG Grid or TanStack Table for the spreadsheet view; ECharts or Recharts; ExcelJS for exports	Shares components, types and API client with the mobile app. A grid component handles the "spreadsheet summary" requirement properly	React SPA with Vite
On-device database	SQLite (op-sqlite) managed by the sync SDK	The tablet reads and writes locally. Taps feel instant and work with no signal	WatermelonDB
Offline sync	**PowerSync** (Postgres-backed, sync rules per site)	Built for Postgres + React Native offline-first. Each tablet only downloads its own site's data, and uploads go through our API so rules are enforced server-side	ElectricSQL; or a custom outbox + delta sync (more work)
Primary database	**PostgreSQL 16+ on Supabase, London region**	Postgres row-level security for multi-tenancy, pgvector for AI search, and managed auth and storage in one UK-hosted bill. Plain Postgres means no lock-in	AWS RDS (eu-west-2) + Cognito + S3 for more control later
API	Node.js + **Hono** (or NestJS) with Zod validation, on Fly.io (London) or AWS ECS (eu-west-2)	Small, fast, typed. It holds the rules engine (pass/fail against nation thresholds), upload validation, contractor links and integrations	Supabase Edge Functions only (harder to test and grow)
Background jobs	**pg-boss** (job queue inside Postgres) + cron	Generates checklist runs from schedules, auto-closes overdue lists, sends end-of-day reports, certificate expiry reminders, FHRS sync and AI batch summaries. No extra vendor	Inngest or Trigger.dev
Auth	Supabase Auth: email magic link + password for managers; Google/Microsoft SSO later. **Kiosk PIN** verified on the device against an argon2 hash	Staff without email can still sign in by PIN on a registered tablet	Clerk
Files & media	Supabase Storage (S3-compatible, London) for photos and documents; **Mux** or Cloudflare Stream for how-to videos	Videos need streaming and transcoding for kitchen tablets on weak Wi-Fi	AWS S3 + CloudFront
Notifications	Expo Push; **Twilio** SMS (UK numbers) for contractor work-order links; Postmark for email	Contractors get a no-login link by SMS. Managers get exception-only push notifications	Vonage, Resend
PDF generation	Headless Chromium (Playwright) rendering HTML templates	EHO packs, end-of-day reports and allergen matrices print exactly like the screen	react-pdf
Billing	**Stripe Billing**: GBP, VAT, per-site quantity, monthly rolling	Matches the £0 / £19 / £39 / £59–69 / estate pricing model	Paddle (merchant of record)
AI	**Anthropic Claude API** via the official TypeScript SDK; embeddings from a separate provider (e.g. Voyage AI)	Strong vision for thermometer and photo reading, tool use for Ask-your-logs, document citations for Ask-the-handbook	Claude via Amazon Bedrock or Google Vertex if a specific cloud region is required
Observability	Sentry (crashes and errors), PostHog EU (product analytics), Better Stack (logs, uptime)	Need to see sync failures on real kitchen devices	Datadog
Kiosk lockdown	iPad Guided Access / Apple Business Manager; Android Lock Task mode via an MDM (e.g. Esper, Hexnode)	Keeps the shared tablet in the app	—

Offline-first and the shared kitchen tablet

The tablet must never block on the network. The design follows a few rules:

- **Records are created on the device.** Every record gets a **UUIDv7** generated on the device, so IDs never clash when several offline devices sync later.
- **Writes are local first.** Each write lands in on-device SQLite immediately. The sync layer uploads queued writes in order when the connection returns and shows a "3 records waiting to sync" badge so managers trust it.
- **Two timestamps per record.** `client_recorded_at` is when the person did it, from the device clock. `server_received_at` is the trusted server time. A large gap between them is flagged to managers,

which defeats changing the tablet clock to backdate checks.

- **Pass or fail is worked out twice.** It is evaluated on the device for instant feedback (red, plus a prompt for a corrective action), then **re-evaluated on the server** against the site's rules pack. The server result is authoritative and stored with a frozen copy of the threshold used.
- **Records are append-only.** A correction is a new record pointing at the old one (`supersedes_id`); nothing is edited or deleted. Database triggers enforce this. That makes the record tamper-evident for an EHO.
- **Each tablet syncs only its own site.** Sync rules download only that site's templates, today's runs, recent history and assets, which keeps the local database small.
- **Kiosk PIN login.** Staff tap their name and enter a 4-digit PIN, which is checked offline against an argon2 hash synced to the device. The session auto-locks after inactivity. Every response is stamped with both the person and the device.
- **Media uploads in the background.** Photos are compressed on the device and uploaded by a background task. The record syncs straight away with a placeholder link, and the photo follows.
- **AI features need a connection; logging does not.** If the tablet is offline, "read the thermometer" falls back to manual entry and queues the photo for checking later.

AI layer design

Model and API choices. Use the official Anthropic TypeScript SDK (`@anthropic-ai/sdk`) from the AI service only; devices never hold API keys. Default to `claude-opus-5-5`. Once you have usage data, evaluate whether a cheaper model holds quality on high-volume routes such as translation or classification, and decide per route. The Claude API features each AI capability relies on:

Capability	Claude API feature used	Notes
Ask your logs (search and analysis)	**Tool use with strict schemas**: the model chooses from the app's own query tools rather than writing SQL	Tools run on the server **as the asking user**, so Postgres row-level security limits what they can see. Every tool returns record IDs, which become clickable citations
Ask the handbook	Document input with **citations enabled** on the operator's approved policies and SFBB safe methods	Answers quote the source passage. If nothing relevant is found, the assistant says so
Read the thermometer, snap-to-setup, fault photos, invoices, allergen labels	**Vision** (image and PDF input) + **structured outputs** (`output_config.format`) for a guaranteed JSON shape	Extracted data is always a *suggestion* a person confirms
End-of-day and weekly summaries, nightly early warnings	**Message Batches API**	Asynchronous and half price; reports aren't time-critical to the minute
Cost control	**Prompt caching** of the fixed system prompt, tool definitions and the site's handbook	Plus a monthly AI quota per organisation (`organisations.ai_monthly_quota`), metered from `ai_interactions`
Robustness	Check `stop_reason` (including `refusal`) before using output; retry 429/5xx with backoff	Show "couldn't answer, here's the manual search" rather than failing silently

Ask-your-logs tool set (first version). Each tool is a typed server function with strict JSON schemas:

Tool	What it does
`search_records(entity, filters, date_range, site_ids, text?)`	Finds checks, responses, corrective actions, work orders, invoices, certificates and visitor entries
`aggregate(metric, group_by, filters, date_range)`	Counts, rates, sums and averages, for example the missed-closing rate by site and weekday
`timeseries(subject, interval, date_range)`	Temperature trend for one fridge, or failures per week

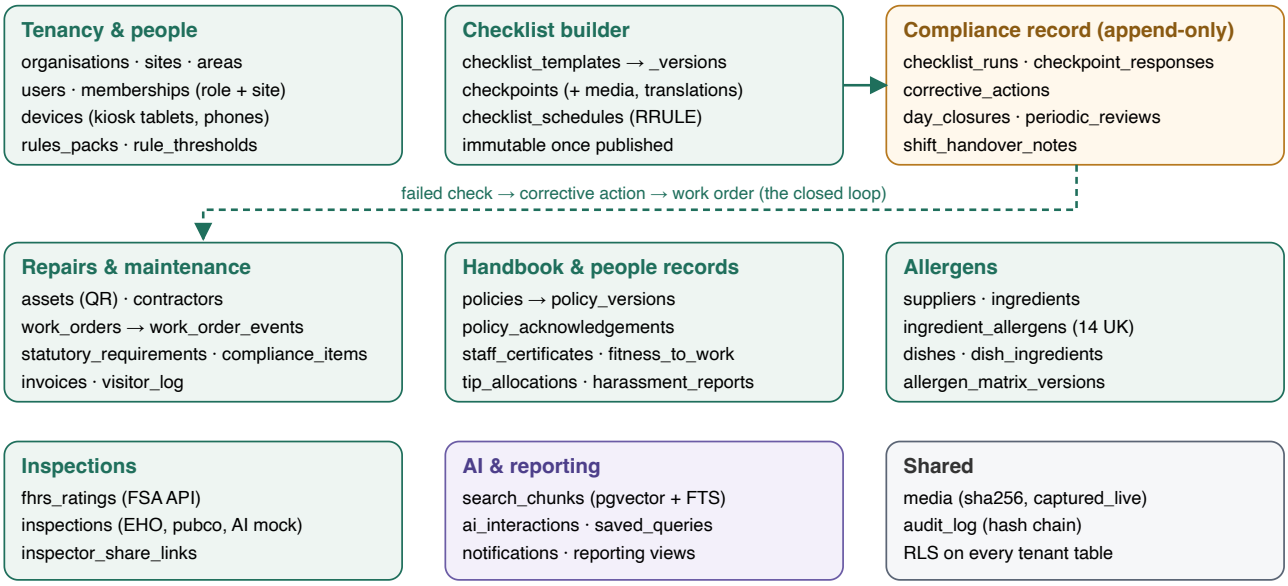
Tool	What it does
`semantic_search(query, sources)`	pgvector plus full-text search across notes, handover logs, policy text and photo captions
`get_record(entity, id)`	Fetches one record with its photos and history
`render_chart(spec)`	Returns a chart spec the console and phone draw natively
`save_query(question, query_spec, schedule?)`	Turns a good question into a scheduled report (`saved_queries` table)

Traceability. Every AI call writes an `ai_interactions` row: feature, model, tools called with their parameters, records cited, output, whether the person accepted or edited it, and token usage.

Data protection. Sign Anthropic's data processing terms and check what processing locations and retention options they offer; don't assume UK-only processing. If an enterprise customer needs AI processing pinned to a specific cloud region, Claude is also available through Amazon Bedrock and Google Vertex AI; confirm which regions those offer for the models you use. Send the model only the fields a feature needs. Never send `fitness_to_work_records` or `harassment_reports` to the model.

Database schema

Domain map



Core tables by domain

Domain	Tables	Key design points
Tenancy & people	`organisations` , `sites` , `areas` , `users` , `memberships` , `devices`	`sites.nation` (England/Wales/Scotland/NI) selects the rules pack. `memberships` holds role + optional site scope (NULL = all sites), which supports your Staff / Manager / Admin / Finance roles plus Owner/Area Manager and read-only Inspector. `users.pin_hash` supports kiosk login; email is optional for staff
Rules packs	`rules_packs` , `rule_thresholds` , `site_threshold_overrides`	Legal thresholds (8°C, 63°C, 82°C reheat in Scotland, cooling times) are **data with a `legal_basis` and a `verified` flag**, versioned and consultant-reviewed. Sites may only set *stricter* house standards

Domain	Tables	Key design points
Checklist builder	`checklist_templates`, `checklist_template_versions`, `checkpoints`, `checkpoint_media`, `checkpoint_translations`, `checklist_schedules`	Your **customisable checkpoints**: any response type (yes/no, temperature, number, photo, signature, select), each with **guidance text and images/videos**. Published versions are immutable, so old records always show the question as it was asked. Schedules use iCal RRULEs and can be paused for closures
Compliance record	`checklist_runs`, `checkpoint_responses`, `corrective_actions`, `day_closures`, `periodic_reviews`, `shift_handover_notes`	Responses store the value, pass/fail, **frozen threshold snapshot**, capture source (manual, probe, camera OCR), photo, person, device and both timestamps. `day_closures` stores the **end-of-day report**. `periodic_reviews` stores the SFBB 4-weekly review
Repairs & maintenance	`assets`, `contractors`, `work_orders`, `work_order_events`, `invoices`, `statutory_requirements`, `compliance_items`, `visitor_log`	**The closed loop:** `corrective_actions.work_order_id` and `work_orders.source_response_id` link a failed check to the repair. `compliance_items` is the statutory calendar (gas CP42, TR19, EICR, fire alarm and others) with certificate uploads. `invoices` feeds the **Finance role** and accounting exports
Handbook & people	`policies`, `policy_versions`, `policy_media`, `policy_acknowledgements`, `staff_certificates`, `fitness_to_work_records`, `tip_allocations`, `harassment_reports`	Publishing a new version triggers re-acknowledgement. Signatures are append-only. The sensitive tables are restricted to owner/admin and never sent to AI
Allergens	`suppliers`, `ingredients`, `ingredient_allergens`, `dishes`, `dish_ingredients`, `allergen_matrix_versions`	The 14 UK allergens are an enum. AI-suggested allergens need `confirmed_by` before they count. Each published matrix is a frozen, printable snapshot
Inspections	`fhrs_ratings`, `inspections`, `inspector_share_links`	FHRS ratings are synced from the FSA open-data API. The read-only, time-limited EHO pack link needs no login
AI & reporting	`search_chunks`, `ai_interactions`, `saved_queries`, `notifications`	Hybrid search: Postgres full-text + pgvector (HNSW index). `search_chunks.visible_to_roles` mirrors permissions. `saved_queries` turns Ask-your-logs questions into scheduled reports
Shared	`media`, `audit_log`	`media.sha256` detects a reused photo and `captured_live` marks in-app camera shots, both anti-pencil-whipping signals. `audit_log` is a **hash chain**, so any tampering with history is detectable

Reporting views in the schema feed the spreadsheet summary and the Ask-your-logs tools. `v_current_responses` takes the latest correction. `v_daily_site_summary` gives one row per site per day: the manager grid. `v_asset_spend` gives cost per asset for Finance. At scale, materialise these views or move analytics to a read replica.

Multi-tenancy and permissions

Every tenant table carries `organisation_id`. The API sets `app.org_id` and `app.user_id` per transaction, and **Postgres row-level security** policies filter by organisation and by `visible_site_ids()`, the sites the user's memberships allow. Role checks for actions (who can sign off a corrective action or approve an invoice) live in the API. A shared permission map mirrors the role matrix in the product plan. Because Ask-your-logs tools run under the same database session, **the AI can never see more than the person asking**.

Retention (to confirm with a solicitor)

Data	Suggested retention	Basis
Food-safety records (responses, corrective actions, reviews)	2 years minimum, configurable up to 6	SFBB requires the diary to be kept until the next inspection; the longer period is a defence against claims **[unverified]**
Holiday and pay-related records	6 years	ERA 2025 holiday-records duty (from April 2026)
Tip allocation records	3 years	Tipping Act 2023 **[unverified period]**
Statutory certificates (gas, EICR, TR19…)	Life of the asset + 2 years	Evidence for insurers and the EHO
Fitness-to-work and harassment records	Minimal; reviewed annually	UK GDPR data minimisation
AI interaction logs	12 months	Debugging and audit; trimmed to essentials

Deletion of personal data (for example a leaver's account) **pseudonymises the user record** rather than deleting it, so compliance records stay intact.

Trading / EOD finance module (replacing the current form)

Maginhawa's finance team currently uses a single static page on Netlify. It calculates totals in the browser and posts JSON to a **public Power Automate URL**, which appends a row to an **Excel table on SharePoint**. Two further flows look up daily targets in Excel and send emails. A code review of that page found five problems:

- The Power Automate signature is exposed in the page source, and nobody has to log in.
- **Net sales ignores discounts and comps**, even though the form says it includes them.
- Delivery net is hard-coded as `gross ÷ 1.2 × 0.55`, and the wage target is hard-coded at 35%.
- Lunch managers are dropped from the submission.
- Discounts, comps, voids and refunds are flattened into text strings that cannot be analysed.

The new module keeps everything useful in it, including the **23 discount/comp reasons**, the **15 W/NW void codes** and the staff positions, and fixes the rest:

Area	Old form	New module
Identity	Typed "Submitted by" / "Approved by" names	Logged-in user; approvers are real users; audit log
Calculations	In the browser; net omits discounts	On the server; `net_sales` is a generated column (gross – service charge – tax – discounts); totals in `v_trading_day_summary`
Settings	20% VAT, 55% delivery retention, 35%/30% wage thresholds hard-coded	`site_finance_settings` and `delivery_platforms` per site/platform; `site_targets` per day
Line items	Text blobs	`trading_adjustments` rows with `reason_codes`, amounts, approver, time
Cash	Warning only; variance not stored	`cash_reconciliations.variance` generated; database refuses a variance without an explanation
Delivery	Estimated net only	Estimated net **and** `actual_payout` from platform statements; refunds with status
Staff & wages	Names in text; one manual wage figure	`shift_staff` rows (user or name, position, hours, cost); wage cost from rota when integrated
Duplicates	Same site and date can be sent twice	`UNIQUE (site_id, business_date)`; drafts saved, then submitted, then approved by finance
Maintenance notes	Free text in an email	`trading_feedback.work_order_id` links to a real work order
Excel	The only store	Synced copy via **Microsoft Graph** (`integration_connections`, idempotent `outbound_sync_jobs`)

Migration (see diagrams 4–6):

1. **Stage 1:** the app replaces the form and writes the same row into finance's workbook, so Flows 2 and 3 keep working.
2. **Stage 2:** the app becomes the source of truth, with targets imported from Excel and a finance dashboard.
3. **Stage 3:** EPOS, rota and delivery integrations fill the report automatically, and scheduled reports replace the Power Automate emails.

Immediate action, independent of the app: regenerate the exposed Power Automate trigger URL.

Microsoft 365 integration adds Microsoft Graph (application permissions scoped to the one workbook or site, e.g. `Sites.Selected`) and optional "Sign in with Microsoft" for the finance team. Credentials live in the secret manager and are referenced by `integration_connections.secret_ref`.

Security, privacy and UK compliance for the platform itself

- **Data location:** host the database, storage and backups in UK (London) regions, with EU fallback documented. Register with the **ICO** as a data controller and publish a DPA and sub-processor list for customers.
- **Certifications:** get **Cyber Essentials** before the first paid customer (pub companies and insurers ask for it). Aim for ISO 27001 in phase 3 for estates and enterprise.
- **Encryption:** TLS everywhere; encryption at rest from the providers. The device SQLite database is encrypted (SQLCipher), and the device can be revoked remotely via `devices.revoked_at`.
- **Backups:** point-in-time recovery on Postgres, plus nightly logical backups to a second UK provider, with restores tested quarterly.
- **Secrets:** API keys (Anthropic, Twilio, Stripe) live only in the server's secret manager, never in apps.
- **Contractor links:** single-use or expiring tokens scoped to one work order.

Engineering workflow

- **Environments:** local (Supabase CLI + Expo dev client) → staging (London) → production. Each pull request gets a preview of the web console.
- **CI:** GitHub Actions to lint, type-check, run unit tests (Vitest) and database tests (pgTAP for RLS policies, triggers and views), then build the Expo app and the web app.
- **Testing priorities:**
- the rules engine (threshold edge cases per nation)
- offline sync conflicts (two devices, same run)
- RLS isolation between organisations and sites
- append-only triggers
- evaluations for AI features: a fixed set of questions and photos with expected answers, re-run whenever prompts or models change
- **Releases:** Expo OTA updates for JavaScript-only fixes, and store releases for native changes. Database migrations use a migration tool (e.g. the Supabase CLI or dbmate), with forward-only migrations.

Build order mapped to the MVP

Sprint block (2 weeks each)	Build
1–2	Monorepo, auth, organisations, sites, memberships, the rules-pack model, the UK template library seeded
3–4	Checklist builder (desktop), tablet kiosk with PIN, offline sync, responses + server-side pass/fail, photo capture
5–6	Corrective actions, schedules + auto-close, manager phone app with exception push notifications, end-of-day report, spreadsheet grid + Excel export
7–8	Assets + QR, fault reports → work orders, contractor SMS links, EHO pack PDF + inspector link
9–10	Handbook builder, e-signed acknowledgements, staff certificates; AI: snap-to-setup, read-the-thermometer, Ask the handbook
7–10 (parallel track)	**Trading / EOD module**: trading days and sessions, reason codes, cash reconciliation, delivery, staff, targets, Excel sync via Graph (Stage 1), finance dashboard (Stage 2)
11–12	**Ask your logs** (tools + citations + saved queries), AI end-of-day summary, Stripe billing, Cyber Essentials, pilot launch

Phase 2 adds the statutory calendar, invoices and Finance exports, the allergen module, EPOS and rota integrations, and Bluetooth probes, in line with the module plan in the product report.

Decisions to confirm

1. **Supabase or raw AWS.** Supabase is faster to build on now. Raw AWS gives more control for later enterprise deals. Because everything is plain Postgres, moving later is feasible.
2. **PowerSync or custom sync.** PowerSync saves months but is a paid dependency. Prototype it with one tablet in a real kitchen basement before committing.
3. **Embedding provider and dimension.** Anthropic doesn't offer an embeddings model. Choose one (e.g. Voyage AI) and set the `vector(...)` dimension in `search_chunks` to match.
4. **AI data processing terms.** Confirm Anthropic's (or a cloud provider's) processing locations and retention options against UK GDPR expectations before pilots.
5. **Threshold verification.** Every `rule_thresholds` and `statutory_requirements` row starts `verified = false`. A food-safety consultant signs them off before launch.

Appendix: full PostgreSQL schema (schema.sql v0.2)

```
-- ====================================================================
-- UK restaurant operations app: PostgreSQL schema (v0.2, Oct 2026 — adds trading/EOD module)
-- Target: PostgreSQL 16+ with pgvector. Multi-tenant (organisation_id
-- on every tenant table) with Row Level Security.
--
-- Conventions
--   * Primary keys are UUIDv7, generated on the client so records can be
--     created offline and synced later without ID clashes.
--   * Compliance records are append-only: a correction is a new row that
--     supersedes the old one; nothing is updated in place or hard-deleted.
--   * Two timestamps on every captured record: client_recorded_at (device
--     clock, when the person did it) and server_received_at (trusted).
--   * Templates are versioned; runs point at the exact version used, so
--     old records always show the question as it was asked.
-- ====================================================================

CREATE EXTENSION IF NOT EXISTS vector;     -- pgvector, for semantic search
CREATE EXTENSION IF NOT EXISTS citext;

-- ---------------------------------------------------------------------
-- Enums
-- ---------------------------------------------------------------------
CREATE TYPE uk_nation        AS ENUM ('england','wales','scotland','northern_ireland');
CREATE TYPE app_role         AS ENUM ('owner','admin','area_manager','manager','staff','finance','inspector_readonly');
CREATE TYPE checklist_kind   AS ENUM
('opening','closing','cleaning','food_safety','delivery','handover','audit','custom');
CREATE TYPE response_type    AS ENUM
('yes_no','number','temperature','text','single_select','multi_select','photo','signature','date','time');
CREATE TYPE run_status       AS ENUM ('scheduled','in_progress','completed','missed','auto_closed','cancelled');
CREATE TYPE capture_source   AS ENUM ('manual','bluetooth_probe','sensor','camera_ocr','import');
CREATE TYPE ca_status        AS ENUM ('open','action_taken','verified','escalated');
CREATE TYPE wo_status        AS ENUM
('reported','triaged','dispatched','scheduled','in_progress','awaiting_parts','resolved','closed','cancelled');
CREATE TYPE wo_priority      AS ENUM ('low','normal','high','urgent');
CREATE TYPE wo_source        AS ENUM ('fault_report','failed_check','statutory','planned','ai_early_warning');
CREATE TYPE media_kind       AS ENUM ('image','video','pdf','audio','other');
CREATE TYPE uk_allergen      AS ENUM ('celery','cereals_gluten','crustaceans','eggs','fish','lupin','milk',
                                      'molluscs','mustard','tree_nuts','peanuts','sesame','soya','sulphites');
CREATE TYPE allergen_presence AS ENUM ('contains','may_contain','free_from');

-- ---------------------------------------------------------------------
-- 1. Tenancy, people and devices
-- ---------------------------------------------------------------------
CREATE TABLE organisations (
  id               uuid PRIMARY KEY,
  name             text NOT NULL,
  legal_name       text,
  companies_house_no text,
  vat_number       text,
  plan             text NOT NULL DEFAULT 'free',          -- free | essentials | operator | group | estate
  stripe_customer_id text,
  ai_monthly_quota int NOT NULL DEFAULT 0,                -- AI usage cap per plan
  created_at       timestamptz NOT NULL DEFAULT now(),
  deleted_at       timestamptz
);

CREATE TABLE sites (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  name             text NOT NULL,
  nation           uk_nation NOT NULL,                    -- drives rules pack (SFBB vs CookSafe, thresholds)
  venue_type       text NOT NULL,                         -- restaurant | pub | cafe | takeaway | hotel_kitchen ...
  address_line1    text, address_line2 text, town text, postcode text,
  timezone         text NOT NULL DEFAULT 'Europe/London',
  fhrs_establishment_id bigint,                           -- FSA open data ID, for rating tracking
  max_capacity     int,                                   -- Martyn's Law tiering (200+/800+)
  day_close_time   time NOT NULL DEFAULT '23:59',         -- when end-of-day report runs
  rules_pack_id    uuid,                                  -- FK added below
  created_at       timestamptz NOT NULL DEFAULT now(),
  deleted_at       timestamptz
);
CREATE INDEX ON sites (organisation_id);

CREATE TABLE areas (                                     -- kitchen, bar, cellar, front of house ...
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
```

```
  site_id          uuid NOT NULL REFERENCES sites(id),
  name             text NOT NULL
);

CREATE TABLE users (
  id               uuid PRIMARY KEY,                       -- = auth provider user id
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  full_name        text NOT NULL,
  email            citext,                                 -- optional: kiosk-only staff may have none
  phone            text,
  preferred_language text NOT NULL DEFAULT 'en-GB',        -- drives AI translation of checkpoints
  pin_hash         text,                                   -- 4-digit kiosk PIN (argon2id), verified offline on device
  employment_start date,
  employment_end   date,
  is_active        boolean NOT NULL DEFAULT true,
  created_at       timestamptz NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX ON users (organisation_id, email) WHERE email IS NOT NULL;

CREATE TABLE memberships (                                 -- role + site scope
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  user_id          uuid NOT NULL REFERENCES users(id),
  role             app_role NOT NULL,
  site_id          uuid REFERENCES sites(id),              -- NULL = all sites in org
  created_at       timestamptz NOT NULL DEFAULT now(),
  UNIQUE (user_id, role, site_id)
);

CREATE TABLE devices (                                     -- registered kiosk tablets & phones
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid REFERENCES sites(id),
  kind             text NOT NULL,                          -- kiosk_tablet | phone | desktop
  label            text,
  platform         text,                                   -- ios | android | web
  registered_by    uuid REFERENCES users(id),
  last_seen_at     timestamptz,
  revoked_at       timestamptz
);

-- ---------------------------------------------------------------------
-- 2. Media (photos, how-to videos, certificates, invoices)
-- ---------------------------------------------------------------------
CREATE TABLE media (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  kind             media_kind NOT NULL,
  storage_key      text NOT NULL,                          -- object storage path (UK region)
  mime_type        text NOT NULL,
  bytes            bigint,
  sha256           text NOT NULL,                          -- detects re-used photos
  captured_live    boolean NOT NULL DEFAULT false,         -- true = taken in-app camera, not gallery
  captured_at      timestamptz,
  captured_by      uuid REFERENCES users(id),
  device_id        uuid REFERENCES devices(id),
  video_playback_id text,                                  -- streaming provider id for how-to videos
  created_at       timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ON media (organisation_id, sha256);

-- ---------------------------------------------------------------------
-- 3. Rules packs (nation-aware thresholds; never hard-coded)
-- ---------------------------------------------------------------------
CREATE TABLE rules_packs (
  id               uuid PRIMARY KEY,
  nation           uk_nation NOT NULL,
  name             text NOT NULL,                          -- e.g. 'SFBB England 2026.1', 'CookSafe 2026.1'
  version          text NOT NULL,
  reviewed_by      text,                                   -- consultant sign-off
  published_at     timestamptz,
  UNIQUE (nation, version)
);
ALTER TABLE sites ADD CONSTRAINT sites_rules_pack_fk FOREIGN KEY (rules_pack_id) REFERENCES rules_packs(id);

CREATE TABLE rule_thresholds (
  id               uuid PRIMARY KEY,
  rules_pack_id    uuid NOT NULL REFERENCES rules_packs(id),
  check_code       text NOT NULL,                          --
'cold_hold','hot_hold','reheat','cook_core','freezer','cooling_stage1'...
  min_value        numeric,
  max_value        numeric,
  unit             text NOT NULL DEFAULT 'C',
```

```
  max_minutes      int,                                  -- for time-bound rules (cooling)
  legal_basis      text,                                 -- citation shown to users / EHO
  verified         boolean NOT NULL DEFAULT false,       -- false until checked against primary source
  UNIQUE (rules_pack_id, check_code)
);

CREATE TABLE site_threshold_overrides (                 -- stricter house standards (never looser)
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  check_code       text NOT NULL,
  min_value        numeric,
  max_value        numeric,
  set_by           uuid NOT NULL REFERENCES users(id),
  created_at       timestamptz NOT NULL DEFAULT now(),
  UNIQUE (site_id, check_code)
);

-- ---------------------------------------------------------------------
-- 4. Checklists & checkpoints (the configurable builder)
-- ---------------------------------------------------------------------
CREATE TABLE checklist_templates (
  id               uuid PRIMARY KEY,
  organisation_id  uuid REFERENCES organisations(id),   -- NULL = system template (UK template library)
  name             text NOT NULL,
  kind             checklist_kind NOT NULL,
  current_version_id uuid,                              -- FK added below
  archived_at      timestamptz,
  created_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE checklist_template_versions (              -- immutable once published
  id               uuid PRIMARY KEY,
  template_id      uuid NOT NULL REFERENCES checklist_templates(id),
  version_no       int NOT NULL,
  published_at     timestamptz,
  published_by     uuid REFERENCES users(id),
  source           text NOT NULL DEFAULT 'manual',      -- manual | template_library | ai_snap_to_setup
  UNIQUE (template_id, version_no)
);
ALTER TABLE checklist_templates ADD CONSTRAINT tmpl_current_version_fk
  FOREIGN KEY (current_version_id) REFERENCES checklist_template_versions(id);

CREATE TABLE checkpoints (
  id               uuid PRIMARY KEY,
  template_version_id uuid NOT NULL REFERENCES checklist_template_versions(id),
  position         int NOT NULL,
  section          text,
  prompt           text NOT NULL,                       -- "Walk-in chiller temperature"
  guidance         text,                                -- "how to do it right" description
  response_type    response_type NOT NULL,
  options          jsonb,                               -- select options
  check_code       text,                                -- links to rule_thresholds for pass/fail
  asset_id         uuid,                                -- optional: checkpoint about a specific asset (FK below)
  required         boolean NOT NULL DEFAULT true,
  photo_required   boolean NOT NULL DEFAULT false,
  corrective_action_required_on_fail boolean NOT NULL DEFAULT true,
  UNIQUE (template_version_id, position)
);

CREATE TABLE checkpoint_media (                         -- how-to images / videos per checkpoint
  checkpoint_id    uuid NOT NULL REFERENCES checkpoints(id),
  media_id         uuid NOT NULL REFERENCES media(id),
  position         int NOT NULL DEFAULT 0,
  PRIMARY KEY (checkpoint_id, media_id)
);

CREATE TABLE checkpoint_translations (                  -- AI-translated, human-reviewable
  checkpoint_id    uuid NOT NULL REFERENCES checkpoints(id),
  language         text NOT NULL,
  prompt           text NOT NULL,
  guidance         text,
  machine_translated boolean NOT NULL DEFAULT true,
  PRIMARY KEY (checkpoint_id, language)
);

CREATE TABLE checklist_schedules (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  template_id      uuid NOT NULL REFERENCES checklist_templates(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  area_id          uuid REFERENCES areas(id),
  rrule            text NOT NULL,                       -- iCal RRULE, e.g. FREQ=DAILY;BYHOUR=8
```

```
  window_minutes   int NOT NULL DEFAULT 60,               -- due window before 'late'
  auto_close       boolean NOT NULL DEFAULT true,
  assigned_role    app_role,
  paused_from      date,                                   -- closures / refurbishment
  paused_until     date,
  created_at       timestamptz NOT NULL DEFAULT now()
);

-- ---------------------------------------------------------------------
-- 5. Runs & responses (the compliance record — append-only)
-- ---------------------------------------------------------------------
CREATE TABLE checklist_runs (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  template_version_id uuid NOT NULL REFERENCES checklist_template_versions(id),
  schedule_id      uuid REFERENCES checklist_schedules(id),
  business_date    date NOT NULL,                          -- trading day (handles past-midnight closes)
  due_at           timestamptz,
  status           run_status NOT NULL DEFAULT 'scheduled',
  started_at       timestamptz,
  completed_at     timestamptz,
  completed_by     uuid REFERENCES users(id),
  device_id        uuid REFERENCES devices(id),
  duration_seconds int,                                    -- pencil-whip signal
  created_at       timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ON checklist_runs (site_id, business_date);
CREATE INDEX ON checklist_runs (organisation_id, status, due_at);

CREATE TABLE checkpoint_responses (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  run_id           uuid NOT NULL REFERENCES checklist_runs(id),
  checkpoint_id    uuid NOT NULL REFERENCES checkpoints(id),
  value_bool       boolean,
  value_num        numeric,
  value_text       text,
  value_json       jsonb,                                  -- selects, signature strokes
  unit             text,
  passed           boolean,                                -- evaluated against threshold at time of capture
  threshold_snapshot jsonb,                                -- min/max/legal_basis used, frozen
  source           capture_source NOT NULL DEFAULT 'manual',
  photo_media_id   uuid REFERENCES media(id),
  ocr_value        numeric,                                -- AI-read thermometer value, for mismatch flags
  recorded_by      uuid NOT NULL REFERENCES users(id),
  device_id        uuid REFERENCES devices(id),
  client_recorded_at timestamptz NOT NULL,
  server_received_at timestamptz NOT NULL DEFAULT now(),
  supersedes_id    uuid REFERENCES checkpoint_responses(id), -- corrections, never updates
  correction_reason text
);
CREATE INDEX ON checkpoint_responses (site_id, client_recorded_at);
CREATE INDEX ON checkpoint_responses (checkpoint_id, client_recorded_at);
CREATE INDEX ON checkpoint_responses (organisation_id, passed) WHERE passed = false;

CREATE TABLE corrective_actions (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  response_id      uuid NOT NULL REFERENCES checkpoint_responses(id),
  action_taken     text NOT NULL,                          -- "moved stock to chiller 2, called engineer"
  food_disposed    boolean,
  status           ca_status NOT NULL DEFAULT 'action_taken',
  recheck_due_at   timestamptz,
  recheck_response_id uuid REFERENCES checkpoint_responses(id),
  work_order_id    uuid,                                   -- the closed loop (FK below)
  recorded_by      uuid NOT NULL REFERENCES users(id),
  verified_by      uuid REFERENCES users(id),
  verified_at      timestamptz,
  created_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE day_closures (                                -- manager "close the day" + end-of-day report
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  business_date    date NOT NULL,
  closed_by        uuid REFERENCES users(id),             -- NULL = auto-closed
  closed_at        timestamptz NOT NULL DEFAULT now(),
  stats            jsonb NOT NULL,                         -- completion %, failures, open CAs, faults
  ai_summary       text,
```

```
  report_media_id  uuid REFERENCES media(id),            -- PDF snapshot
  UNIQUE (site_id, business_date)
);

CREATE TABLE periodic_reviews (                          -- SFBB 4-weekly review / CookSafe equivalent
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  period_start     date NOT NULL,
  period_end       date NOT NULL,
  answers          jsonb NOT NULL,
  ai_draft         text,
  signed_by        uuid REFERENCES users(id),
  signed_at        timestamptz
);

CREATE TABLE shift_handover_notes (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  business_date    date NOT NULL,
  author_id        uuid NOT NULL REFERENCES users(id),
  body             text NOT NULL,
  tags             text[],
  created_at       timestamptz NOT NULL DEFAULT now()
);

-- ---------------------------------------------------------------------
-- 6. Assets, contractors, work orders (repairs & maintenance)
-- ---------------------------------------------------------------------
CREATE TABLE contractors (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  name             text NOT NULL,
  trade            text NOT NULL,                        -- refrigeration, gas, electrical, extraction, pest ...
  contact_name     text, phone text, email citext,
  gas_safe_number  text,
  accreditation    text,                                 -- e.g. BESA TR19 member
  avg_response_hours numeric,                            -- derived
  created_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE assets (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  area_id          uuid REFERENCES areas(id),
  name             text NOT NULL,                        -- "Walk-in chiller"
  category         text NOT NULL,                        -- refrigeration, cooking, extraction, gas, electrical, lift
...
  make text, model text, serial_number text,
  qr_code          text UNIQUE,                          -- sticker on the unit
  install_date     date,
  purchase_cost    numeric(12,2),
  warranty_expires date,
  default_contractor_id uuid REFERENCES contractors(id),
  photo_media_id   uuid REFERENCES media(id),
  retired_at       date
);
ALTER TABLE checkpoints ADD CONSTRAINT checkpoints_asset_fk FOREIGN KEY (asset_id) REFERENCES assets(id);

CREATE TABLE work_orders (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  asset_id         uuid REFERENCES assets(id),
  reference        text NOT NULL,                        -- human-friendly WO-2026-00123
  title            text NOT NULL,
  description      text,
  source           wo_source NOT NULL,
  source_response_id uuid REFERENCES checkpoint_responses(id),
  priority         wo_priority NOT NULL DEFAULT 'normal',
  status           wo_status NOT NULL DEFAULT 'reported',
  reported_by      uuid REFERENCES users(id),
  assigned_contractor_id uuid REFERENCES contractors(id),
  contractor_link_token text UNIQUE,                     -- no-login SMS/email link for contractor
  dispatched_at    timestamptz,
  attended_at      timestamptz,
  resolved_at      timestamptz,
  ai_drafted       boolean NOT NULL DEFAULT false,
  created_at       timestamptz NOT NULL DEFAULT now(),
  UNIQUE (organisation_id, reference)
);
```

```
CREATE INDEX ON work_orders (site_id, status);
CREATE INDEX ON work_orders (asset_id, created_at);
ALTER TABLE corrective_actions ADD CONSTRAINT ca_work_order_fk FOREIGN KEY (work_order_id) REFERENCES work_orders(id);

CREATE TABLE work_order_events (                          -- timeline: status changes, notes, photos
  id               uuid PRIMARY KEY,
  work_order_id    uuid NOT NULL REFERENCES work_orders(id),
  actor_user_id    uuid REFERENCES users(id),
  actor_contractor_id uuid REFERENCES contractors(id),
  event_type       text NOT NULL,                         -- status_change | note | photo | quote | visit
  from_status      wo_status, to_status wo_status,
  body             text,
  media_id         uuid REFERENCES media(id),
  created_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE invoices (                                   -- Finance role
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  contractor_id    uuid REFERENCES contractors(id),
  work_order_id    uuid REFERENCES work_orders(id),
  asset_id         uuid REFERENCES assets(id),
  invoice_number   text,
  invoice_date     date,
  net_amount       numeric(12,2),
  vat_amount       numeric(12,2),
  gross_amount     numeric(12,2),
  currency         char(3) NOT NULL DEFAULT 'GBP',
  document_media_id uuid REFERENCES media(id),
  extracted_by_ai  boolean NOT NULL DEFAULT false,
  approved_by      uuid REFERENCES users(id),
  accounting_system text,                                 -- xero | quickbooks | sage
  accounting_external_id text,
  exported_at      timestamptz,
  created_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE statutory_requirements (                     -- system catalogue: gas, TR19, EICR, fire alarm ...
  id               uuid PRIMARY KEY,
  code             text NOT NULL UNIQUE,                  --
'gas_safety_cp42','tr19_duct_clean','eicr','fire_alarm_weekly'...
  name             text NOT NULL,
  nations          uk_nation[] NOT NULL,
  default_interval interval,                              -- e.g. '12 months'
  legal_basis      text,
  verified         boolean NOT NULL DEFAULT false
);

CREATE TABLE compliance_items (                           -- the maintenance-compliance calendar
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  asset_id         uuid REFERENCES assets(id),
  requirement_id   uuid NOT NULL REFERENCES statutory_requirements(id),
  interval_override interval,
  last_completed_on date,
  next_due_on      date NOT NULL,
  contractor_id    uuid REFERENCES contractors(id),
  certificate_media_id uuid REFERENCES media(id),
  work_order_id    uuid REFERENCES work_orders(id)
);
CREATE INDEX ON compliance_items (organisation_id, next_due_on);

CREATE TABLE visitor_log (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  visitor_name     text NOT NULL,
  company          text,
  purpose          text,
  contractor_id    uuid REFERENCES contractors(id),
  work_order_id    uuid REFERENCES work_orders(id),
  signed_in_at     timestamptz NOT NULL,
  signed_out_at    timestamptz,
  health_declaration jsonb                                -- food-handler fitness questions
);

-- ------------------------------------------------------------------
-- 7. Handbook, policies & people records
-- ------------------------------------------------------------------
CREATE TABLE policies (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
```

```
  title            text NOT NULL,
  category         text NOT NULL,                          -- food_safety, allergens, hr, harassment, tipping, h_and_s
...
  current_version_id uuid,
  applies_to_roles app_role[],
  archived_at      timestamptz
);

CREATE TABLE policy_versions (                             -- immutable once published
  id               uuid PRIMARY KEY,
  policy_id        uuid NOT NULL REFERENCES policies(id),
  version_no       int NOT NULL,
  body_markdown    text NOT NULL,
  summary          text,
  requires_acknowledgement boolean NOT NULL DEFAULT true,
  source           text NOT NULL DEFAULT 'manual',         -- manual | uk_template | ai_draft
  approved_by      uuid REFERENCES users(id),
  published_at     timestamptz,
  UNIQUE (policy_id, version_no)
);
ALTER TABLE policies ADD CONSTRAINT policies_current_version_fk
  FOREIGN KEY (current_version_id) REFERENCES policy_versions(id);

CREATE TABLE policy_media (
  policy_version_id uuid NOT NULL REFERENCES policy_versions(id),
  media_id         uuid NOT NULL REFERENCES media(id),
  PRIMARY KEY (policy_version_id, media_id)
);

CREATE TABLE policy_acknowledgements (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  policy_version_id uuid NOT NULL REFERENCES policy_versions(id),
  user_id          uuid NOT NULL REFERENCES users(id),
  acknowledged_at  timestamptz NOT NULL,
  signature_json   jsonb,
  device_id        uuid REFERENCES devices(id),
  UNIQUE (policy_version_id, user_id)
);

CREATE TABLE staff_certificates (                          -- Level 2 food hygiene, personal licence, first aid ...
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  user_id          uuid NOT NULL REFERENCES users(id),
  cert_type        text NOT NULL,
  issuer           text,
  issued_on        date,
  expires_on       date,
  document_media_id uuid REFERENCES media(id)
);
CREATE INDEX ON staff_certificates (organisation_id, expires_on);

CREATE TABLE fitness_to_work_records (                     -- illness exclusion / return-to-work (sensitive)
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid REFERENCES sites(id),
  user_id          uuid NOT NULL REFERENCES users(id),
  reported_on      date NOT NULL,
  symptoms_category text,                                  -- minimal data; no free-text diagnosis
  excluded_until   date,
  cleared_by       uuid REFERENCES users(id),
  cleared_on       date
);

-- ERA 2025 evidence (phase 2) — kept simple here
CREATE TABLE tip_allocations (                             -- Employment (Allocation of Tips) Act 2023
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  period_start     date NOT NULL,
  period_end       date NOT NULL,
  total_tips       numeric(12,2) NOT NULL,
  allocation       jsonb NOT NULL,                         -- per-worker amounts
  policy_version_id uuid REFERENCES policy_versions(id),
  created_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE harassment_reports (                          -- incl. third-party harassment (restricted access)
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid REFERENCES sites(id),
  reported_by      uuid REFERENCES users(id),             -- NULL allowed for anonymous
  third_party      boolean NOT NULL DEFAULT false,
```

```
  occurred_at      timestamptz,
  description      text NOT NULL,
  actions_taken    text,
  status           text NOT NULL DEFAULT 'open',
  created_at       timestamptz NOT NULL DEFAULT now()
);

-- ---------------------------------------------------------------------
-- 8. Allergens (14 UK allergens, versioned matrix)
-- ---------------------------------------------------------------------
CREATE TABLE suppliers (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  name             text NOT NULL,
  contact          text,
  approved_at      date
);

CREATE TABLE ingredients (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  name             text NOT NULL,
  supplier_id      uuid REFERENCES suppliers(id),
  spec_media_id    uuid REFERENCES media(id),             -- supplier spec sheet / label photo
  spec_checked_at  date
);

CREATE TABLE ingredient_allergens (
  ingredient_id    uuid NOT NULL REFERENCES ingredients(id),
  allergen         uk_allergen NOT NULL,
  presence         allergen_presence NOT NULL,
  ai_suggested     boolean NOT NULL DEFAULT false,
  confirmed_by     uuid REFERENCES users(id),             -- required before it counts
  confirmed_at     timestamptz,
  PRIMARY KEY (ingredient_id, allergen)
);

CREATE TABLE dishes (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  name             text NOT NULL,
  epos_item_id     text,                                   -- menu sync
  is_ppds          boolean NOT NULL DEFAULT false,         -- Natasha's Law labelling
  active           boolean NOT NULL DEFAULT true
);

CREATE TABLE dish_ingredients (
  dish_id          uuid NOT NULL REFERENCES dishes(id),
  ingredient_id    uuid NOT NULL REFERENCES ingredients(id),
  PRIMARY KEY (dish_id, ingredient_id)
);

CREATE TABLE allergen_matrix_versions (                   -- frozen snapshot each time the matrix is published
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid REFERENCES sites(id),
  version_no       int NOT NULL,
  matrix           jsonb NOT NULL,                         -- {dish_id: {allergen: presence}}
  published_by     uuid NOT NULL REFERENCES users(id),
  published_at     timestamptz NOT NULL DEFAULT now(),
  pdf_media_id     uuid REFERENCES media(id)
);

-- ---------------------------------------------------------------------
-- 9. Inspections & FHRS
-- ---------------------------------------------------------------------
CREATE TABLE fhrs_ratings (                               -- synced from FSA open data API
  id               uuid PRIMARY KEY,
  site_id          uuid NOT NULL REFERENCES sites(id),
  rating_value     text NOT NULL,                          -- '0'..'5', or FHIS 'Pass'/'Improvement Required'
  rating_date      date NOT NULL,
  score_hygiene    int, score_structural int, score_confidence_in_management int,
  fetched_at       timestamptz NOT NULL DEFAULT now(),
  UNIQUE (site_id, rating_date)
);

CREATE TABLE inspections (                                -- EHO visits, insurer/pubco audits, mock inspections
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  kind             text NOT NULL,                          -- eho | pubco_audit | insurer | ai_mock
  visited_on       date NOT NULL,
  inspector_name   text,
```

```
  outcome          text,
  findings         jsonb,
  report_media_id  uuid REFERENCES media(id),
  action_plan      jsonb
);

CREATE TABLE inspector_share_links (                    -- read-only EHO pack link
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  token            text NOT NULL UNIQUE,
  date_from        date NOT NULL,
  date_to          date NOT NULL,
  created_by       uuid NOT NULL REFERENCES users(id),
  expires_at       timestamptz NOT NULL,
  revoked_at       timestamptz
);

-- ------------------------------------------------------------------
-- 10. Notifications, saved questions & reports
-- ------------------------------------------------------------------
CREATE TABLE notifications (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  user_id          uuid NOT NULL REFERENCES users(id),
  kind             text NOT NULL,                       -- missed_check | failed_check | cert_expiring | wo_update
...
  payload          jsonb NOT NULL,
  channel          text NOT NULL,                       -- push | email | sms | in_app
  sent_at          timestamptz,
  read_at          timestamptz
);

CREATE TABLE saved_queries (                            -- "Ask your logs" questions saved as reports
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  owner_id         uuid NOT NULL REFERENCES users(id),
  question         text NOT NULL,                       -- original natural-language question
  query_spec       jsonb NOT NULL,                      -- validated structured query the AI produced
  visualisation    text,                                -- table | bar | line | number
  schedule_rrule   text,                                -- e.g. every Monday 07:00
  deliver_to       uuid[],
  pinned_to_dashboard boolean NOT NULL DEFAULT false,
  created_at       timestamptz NOT NULL DEFAULT now()
);

-- ------------------------------------------------------------------
-- 11. AI: semantic index, interaction log, usage metering
-- ------------------------------------------------------------------
CREATE TABLE search_chunks (                            -- notes, handover text, policy text, photo captions
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid REFERENCES sites(id),
  source_table     text NOT NULL,
  source_id        uuid NOT NULL,
  visible_to_roles app_role[] NOT NULL,                -- enforced at query time, mirrors RLS
  content          text NOT NULL,
  content_tsv      tsvector GENERATED ALWAYS AS (to_tsvector('english', content)) STORED,
  embedding        vector(1024),                        -- dimension depends on embedding model chosen
  created_at       timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ON search_chunks USING gin (content_tsv);
CREATE INDEX ON search_chunks USING hnsw (embedding vector_cosine_ops);
CREATE INDEX ON search_chunks (organisation_id, source_table);

CREATE TABLE ai_interactions (                          -- every AI call is traceable
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  user_id          uuid REFERENCES users(id),
  feature          text NOT NULL,                       -- ask_logs | ask_handbook | snap_setup | ocr_temp |
fault_draft | eod_summary ...
  model            text NOT NULL,
  input_summary    text,
  tool_calls       jsonb,                               -- which query tools ran, with params
  cited_records    jsonb,                               -- [{table, id}] shown to the user
  output           text,
  accepted         boolean,                             -- did the human accept / edit / reject
  input_tokens     int, output_tokens int, cache_read_tokens int,
  latency_ms       int,
  created_at       timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ON ai_interactions (organisation_id, created_at);
```

```
-- ------------------------------------------------------------------
-- 12. Audit log (tamper-evident, append-only)
-- ------------------------------------------------------------------
CREATE TABLE audit_log (
  id              bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  organisation_id  uuid NOT NULL,
  actor_user_id    uuid,
  action           text NOT NULL,                        -- insert | supersede | publish | export | login ...
  entity_table     text NOT NULL,
  entity_id        uuid,
  diff             jsonb,
  occurred_at      timestamptz NOT NULL DEFAULT now(),
  prev_hash        bytea,
  row_hash         bytea NOT NULL                        -- sha256(prev_hash || row) — hash chain
);
CREATE INDEX ON audit_log (organisation_id, occurred_at);

-- Block UPDATE/DELETE on compliance records (corrections use supersedes_id)
CREATE OR REPLACE FUNCTION forbid_mutation() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  RAISE EXCEPTION 'Table % is append-only', TG_TABLE_NAME;
END $$;

CREATE TRIGGER checkpoint_responses_append_only BEFORE UPDATE OR DELETE ON checkpoint_responses
  FOR EACH ROW EXECUTE FUNCTION forbid_mutation();
CREATE TRIGGER policy_acks_append_only BEFORE UPDATE OR DELETE ON policy_acknowledgements
  FOR EACH ROW EXECUTE FUNCTION forbid_mutation();
CREATE TRIGGER audit_log_append_only BEFORE UPDATE OR DELETE ON audit_log
  FOR EACH ROW EXECUTE FUNCTION forbid_mutation();

-- ------------------------------------------------------------------
-- 13. Row Level Security (pattern — apply to every tenant table)
-- ------------------------------------------------------------------
-- The API sets these per request/transaction:
--   SET LOCAL app.org_id = '<uuid>';  SET LOCAL app.user_id = '<uuid>';
CREATE OR REPLACE FUNCTION current_org() RETURNS uuid LANGUAGE sql STABLE AS
  $$ SELECT nullif(current_setting('app.org_id', true), '')::uuid $$;
CREATE OR REPLACE FUNCTION current_app_user() RETURNS uuid LANGUAGE sql STABLE AS
  $$ SELECT nullif(current_setting('app.user_id', true), '')::uuid $$;

-- Sites the current user can see (NULL site_id membership = all sites)
CREATE OR REPLACE FUNCTION visible_site_ids() RETURNS SETOF uuid LANGUAGE sql STABLE AS $$
  SELECT s.id FROM sites s
  WHERE s.organisation_id = current_org()
    AND EXISTS (SELECT 1 FROM memberships m
                WHERE m.user_id = current_app_user()
                  AND (m.site_id IS NULL OR m.site_id = s.id))
$$;

ALTER TABLE sites                ENABLE ROW LEVEL SECURITY;
ALTER TABLE checklist_runs       ENABLE ROW LEVEL SECURITY;
ALTER TABLE checkpoint_responses ENABLE ROW LEVEL SECURITY;
ALTER TABLE work_orders          ENABLE ROW LEVEL SECURITY;
ALTER TABLE invoices             ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_sites ON sites
  USING (organisation_id = current_org());
CREATE POLICY site_scope_runs ON checklist_runs
  USING (organisation_id = current_org() AND site_id IN (SELECT visible_site_ids()));
CREATE POLICY site_scope_responses ON checkpoint_responses
  USING (organisation_id = current_org() AND site_id IN (SELECT visible_site_ids()));
CREATE POLICY site_scope_work_orders ON work_orders
  USING (organisation_id = current_org() AND site_id IN (SELECT visible_site_ids()));
CREATE POLICY finance_invoices ON invoices
  USING (organisation_id = current_org() AND EXISTS (
    SELECT 1 FROM memberships m WHERE m.user_id = current_app_user()
      AND m.role IN ('owner','admin','finance','area_manager','manager')));
-- Repeat the tenant + site-scope pattern for remaining tables; restrict
-- fitness_to_work_records and harassment_reports to owner/admin only.

-- ------------------------------------------------------------------
-- 14. Reporting views (feed the spreadsheet summary & Ask-your-logs tools)
-- ------------------------------------------------------------------
CREATE VIEW v_current_responses AS                       -- latest correction wins
SELECT r.* FROM checkpoint_responses r
WHERE NOT EXISTS (SELECT 1 FROM checkpoint_responses n WHERE n.supersedes_id = r.id);

CREATE VIEW v_daily_site_summary AS                      -- one row per site per day: the spreadsheet grid
SELECT cr.organisation_id, cr.site_id, cr.business_date,
       count(*)                                          AS runs_total,
       count(*) FILTER (WHERE cr.status = 'completed')   AS runs_completed,
       count(*) FILTER (WHERE cr.status IN ('missed','auto_closed')) AS runs_missed,
       (SELECT count(*) FROM v_current_responses r
```

```
         JOIN checklist_runs x ON x.id = r.run_id
         WHERE x.site_id = cr.site_id AND x.business_date = cr.business_date
           AND r.passed = false)                          AS failed_checks
FROM checklist_runs cr
GROUP BY cr.organisation_id, cr.site_id, cr.business_date;

CREATE VIEW v_asset_spend AS                               -- Finance: cost per asset
SELECT a.organisation_id, a.site_id, a.id AS asset_id, a.name, a.category,
       count(DISTINCT w.id)             AS work_orders,
       coalesce(sum(i.gross_amount), 0)  AS total_spend_gbp,
       max(w.created_at)                 AS last_fault_at
FROM assets a
LEFT JOIN work_orders w ON w.asset_id = a.id
LEFT JOIN invoices i    ON i.work_order_id = w.id
GROUP BY a.organisation_id, a.site_id, a.id, a.name, a.category;

-- ================================================================
-- 15. Trading / EOD finance module (replaces the Netlify + Power
--     Automate EOD form). Structured line items instead of text blobs,
--     server-side calculations, real users as approvers, targets per site.
-- ================================================================
CREATE TYPE trading_session_kind AS ENUM ('lunch','dinner','all_day');
CREATE TYPE trading_day_status   AS ENUM ('draft','submitted','approved','locked','reopened');
CREATE TYPE adjustment_kind      AS ENUM ('discount','comp','void');
CREATE TYPE adjustment_scope     AS ENUM ('check','item');
CREATE TYPE refund_status        AS ENUM ('yes','no','pending');
CREATE TYPE staff_department     AS ENUM ('kitchen','foh','management');
CREATE TYPE feedback_category    AS ENUM ('service','kitchen','staff','complaints','maintenance');

CREATE TABLE site_finance_settings (                       -- replaces values hard-coded in the old form
  site_id          uuid PRIMARY KEY REFERENCES sites(id),
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  vat_rate_pct     numeric(5,2) NOT NULL DEFAULT 20,
  wage_target_pct  numeric(5,2) NOT NULL DEFAULT 35,       -- red above this
  wage_amber_pct   numeric(5,2) NOT NULL DEFAULT 30,       -- amber above this
  net_sales_rule   text NOT NULL DEFAULT 'gross_minus_sc_tax_discounts', -- confirm with finance
  sessions         trading_session_kind[] NOT NULL DEFAULT '{lunch,dinner}'
);

CREATE TABLE site_targets (                                -- imported from finance's Excel or edited in console
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  target_date      date NOT NULL,
  gross_sales_target numeric(12,2),
  net_sales_target numeric(12,2),
  covers_target    int,
  asph_target      numeric(8,2),
  wage_pct_target  numeric(5,2),
  source           text NOT NULL DEFAULT 'manual',         -- manual | excel_import
  imported_at      timestamptz,
  UNIQUE (site_id, target_date)
);

CREATE TABLE reason_codes (                                -- house policy lists (discount/comp/void reasons)
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  kind             adjustment_kind NOT NULL,               -- discount & comp share codes in the old form; kept
separate here
  label            text NOT NULL,                          -- 'Employee Dining up to 4 Food Only'
  scope            adjustment_scope,                       -- CHECK or ITEM
  default_pct      numeric(5,2),                           -- 20, 50, 100; NULL = chosen amount
  is_waste         boolean,                                -- voids: W (waste) vs NW (non-waste)
  applies_to       text,                                   -- FOOD | DRINKS | FOOD/DRINK
  note_prompt      text,                                   -- 'Name and site name of the employee'
  note_required    boolean NOT NULL DEFAULT true,
  approval_role    app_role DEFAULT 'manager',
  sort_order       int NOT NULL DEFAULT 0,
  active           boolean NOT NULL DEFAULT true,
  UNIQUE (organisation_id, kind, label)
);

CREATE TABLE staff_positions (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  department       staff_department NOT NULL,
  name             text NOT NULL,                          -- Head chef, CDP, Runner, Duty manager ...
  sort_order       int NOT NULL DEFAULT 0,
  UNIQUE (organisation_id, department, name)
);

CREATE TABLE trading_days (                                -- one per site per service date (no duplicates)
  id               uuid PRIMARY KEY,
```

```
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  site_id          uuid NOT NULL REFERENCES sites(id),
  business_date    date NOT NULL,
  status           trading_day_status NOT NULL DEFAULT 'draft',
  wage_cost        numeric(12,2),
  wage_cost_source text NOT NULL DEFAULT 'manual',         -- manual | rota_integration
  data_source      text NOT NULL DEFAULT 'manual',         -- manual | epos_import
  next_shift_notes text,
  submitted_by     uuid REFERENCES users(id),
  submitted_at     timestamptz,
  approved_by      uuid REFERENCES users(id),             -- finance sign-off
  approved_at      timestamptz,
  day_closure_id   uuid REFERENCES day_closures(id),      -- links trading + ops end-of-day
  created_at       timestamptz NOT NULL DEFAULT now(),
  updated_at       timestamptz NOT NULL DEFAULT now(),
  UNIQUE (site_id, business_date)
);
CREATE INDEX ON trading_days (organisation_id, business_date);

CREATE TABLE trading_sessions (
  id               uuid PRIMARY KEY,
  trading_day_id   uuid NOT NULL REFERENCES trading_days(id) ON DELETE CASCADE,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  session          trading_session_kind NOT NULL,
  gross_sales      numeric(12,2) NOT NULL DEFAULT 0 CHECK (gross_sales >= 0),
  service_charge   numeric(12,2) NOT NULL DEFAULT 0 CHECK (service_charge >= 0),
  tax              numeric(12,2) NOT NULL DEFAULT 0 CHECK (tax >= 0),
  discounts_comps_total numeric(12,2) NOT NULL DEFAULT 0 CHECK (discounts_comps_total >= 0), -- from EPOS; reconciled
to line items
  net_sales        numeric(12,2) GENERATED ALWAYS AS
                     (greatest(gross_sales - service_charge - tax - discounts_comps_total, 0)) STORED,
  food_sales       numeric(12,2) NOT NULL DEFAULT 0,
  drink_sales      numeric(12,2) NOT NULL DEFAULT 0,
  booked_covers    int NOT NULL DEFAULT 0 CHECK (booked_covers >= 0),   -- "starting covers" in old form
  walkins          int NOT NULL DEFAULT 0 CHECK (walkins >= 0),
  covers           int GENERATED ALWAYS AS (booked_covers + walkins) STORED,
  UNIQUE (trading_day_id, session)
);

CREATE TABLE trading_adjustments (                        -- discounts, comps and voids as real rows
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  trading_session_id uuid NOT NULL REFERENCES trading_sessions(id) ON DELETE CASCADE,
  kind             adjustment_kind NOT NULL,
  reason_code_id   uuid NOT NULL REFERENCES reason_codes(id),
  guest_or_staff_name text,
  items            text,
  sale_amount      numeric(12,2),                         -- check/item value before adjustment
  amount           numeric(12,2) NOT NULL CHECK (amount >= 0),
  occurred_at      time,
  performed_by     uuid REFERENCES users(id),             -- "voided by"
  approved_by      uuid REFERENCES users(id),             -- real user, not free text
  notes            text,
  created_at       timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ON trading_adjustments (organisation_id, kind, reason_code_id);

CREATE TABLE cash_reconciliations (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  trading_session_id uuid NOT NULL UNIQUE REFERENCES trading_sessions(id) ON DELETE CASCADE,
  cash_in_epos     numeric(12,2) NOT NULL DEFAULT 0,
  cash_collected   numeric(12,2) NOT NULL DEFAULT 0,
  variance         numeric(12,2) GENERATED ALWAYS AS (cash_collected - cash_in_epos) STORED,
  explanation      text,
  counted_by       uuid REFERENCES users(id),
  checked_by       uuid REFERENCES users(id),
  CONSTRAINT variance_needs_explanation
    CHECK (abs(cash_collected - cash_in_epos) <= 0.01 OR coalesce(length(trim(explanation)),0) > 0)
);

CREATE TABLE delivery_platforms (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  name             text NOT NULL,                         -- Deliveroo, Uber Eats, Just Eat
  estimated_net_pct numeric(5,2) NOT NULL DEFAULT 55,     -- old form hard-coded gross/1.2*0.55
  prices_include_vat boolean NOT NULL DEFAULT true,
  active           boolean NOT NULL DEFAULT true,
  UNIQUE (organisation_id, name)
);

CREATE TABLE delivery_sales (
  id               uuid PRIMARY KEY,
```

```
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  trading_day_id   uuid NOT NULL REFERENCES trading_days(id) ON DELETE CASCADE,
  platform_id      uuid NOT NULL REFERENCES delivery_platforms(id),
  gross_sales      numeric(12,2) NOT NULL DEFAULT 0,
  estimated_net    numeric(12,2),                          -- computed by API from platform settings
  actual_payout    numeric(12,2),                          -- filled later from platform statement
  payout_reference text,
  current_promotion text,
  UNIQUE (trading_day_id, platform_id)
);

CREATE TABLE delivery_refunds (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  trading_day_id   uuid NOT NULL REFERENCES trading_days(id) ON DELETE CASCADE,
  platform_id      uuid NOT NULL REFERENCES delivery_platforms(id),
  amount           numeric(12,2) NOT NULL,
  request_details  text NOT NULL,
  issued           refund_status NOT NULL DEFAULT 'pending'
);

CREATE TABLE shift_staff (                                 -- who worked each session
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  trading_session_id uuid NOT NULL REFERENCES trading_sessions(id) ON DELETE CASCADE,
  user_id          uuid REFERENCES users(id),
  staff_name       text,                                   -- fallback for staff without accounts
  position_id      uuid NOT NULL REFERENCES staff_positions(id),
  hours            numeric(5,2),
  wage_cost        numeric(10,2),                          -- from rota integration when available
  CHECK (user_id IS NOT NULL OR staff_name IS NOT NULL)
);

CREATE TABLE trading_feedback (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  trading_day_id   uuid NOT NULL REFERENCES trading_days(id) ON DELETE CASCADE,
  category         feedback_category NOT NULL,
  body             text NOT NULL,
  work_order_id    uuid REFERENCES work_orders(id),        -- "urgent maintenance" becomes a real job
  created_by       uuid REFERENCES users(id),
  created_at       timestamptz NOT NULL DEFAULT now()
);

-- Integrations & outbound sync (Excel/SharePoint, Power Automate, EPOS)
CREATE TABLE integration_connections (
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  provider         text NOT NULL,                          -- microsoft_graph | power_automate | epos_now | square |
lightspeed | deputy | xero ...
  config           jsonb NOT NULL DEFAULT '{}',            -- e.g. workbook id, table name, column map
  secret_ref       text,                                   -- pointer to secret manager, never the secret
  status           text NOT NULL DEFAULT 'active',
  created_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE outbound_sync_jobs (                          -- idempotent push of each submitted day to Excel etc.
  id               uuid PRIMARY KEY,
  organisation_id  uuid NOT NULL REFERENCES organisations(id),
  connection_id    uuid NOT NULL REFERENCES integration_connections(id),
  entity_table     text NOT NULL,
  entity_id        uuid NOT NULL,
  payload_hash     text NOT NULL,
  status           text NOT NULL DEFAULT 'queued',         -- queued | sent | failed
  attempts         int NOT NULL DEFAULT 0,
  last_error       text,
  sent_at          timestamptz,
  UNIQUE (connection_id, entity_table, entity_id, payload_hash)
);

ALTER TABLE trading_days        ENABLE ROW LEVEL SECURITY;
ALTER TABLE trading_adjustments ENABLE ROW LEVEL SECURITY;
CREATE POLICY site_scope_trading_days ON trading_days
  USING (organisation_id = current_org() AND site_id IN (SELECT visible_site_ids()));
CREATE POLICY tenant_trading_adjustments ON trading_adjustments
  USING (organisation_id = current_org());

-- Day summary: the corrected version of the old form's "Combined day totals"
CREATE VIEW v_trading_day_summary AS
WITH s AS (
  SELECT trading_day_id,
         sum(gross_sales) AS dine_gross, sum(net_sales) AS dine_net,
         sum(food_sales) AS food, sum(drink_sales) AS drink, sum(covers) AS covers
```

```
  FROM trading_sessions GROUP BY trading_day_id),
d AS (
  SELECT trading_day_id, sum(gross_sales) AS del_gross,
         sum(coalesce(actual_payout, estimated_net)) AS del_net
  FROM delivery_sales GROUP BY trading_day_id)
SELECT td.organisation_id, td.site_id, td.business_date, td.status,
       coalesce(s.dine_gross,0) + coalesce(d.del_gross,0)        AS total_gross,
       coalesce(s.dine_net,0)   + coalesce(d.del_net,0)          AS total_net,
       s.dine_net                                                AS dine_in_net,
       s.covers                                                  AS total_covers,
       round(s.dine_net / nullif(s.covers,0), 2)                 AS asph_net,
       round(s.food  / nullif(s.dine_net,0) * 100, 1)            AS food_pct_of_net,
       round(s.drink / nullif(s.dine_net,0) * 100, 1)            AS drink_pct_of_net,
       td.wage_cost,
       round(td.wage_cost / nullif(coalesce(s.dine_net,0)+coalesce(d.del_net,0),0) * 100, 1) AS wage_pct,
       coalesce(t.wage_pct_target, fs.wage_target_pct)           AS wage_pct_target,
       t.net_sales_target,
       (coalesce(s.dine_net,0)+coalesce(d.del_net,0)) - t.net_sales_target AS net_vs_target
FROM trading_days td
LEFT JOIN s  ON s.trading_day_id = td.id
LEFT JOIN d  ON d.trading_day_id = td.id
LEFT JOIN site_targets t ON t.site_id = td.site_id AND t.target_date = td.business_date
LEFT JOIN site_finance_settings fs ON fs.site_id = td.site_id;
```